Agent Security Policy: Enforcing Integrity and Confidentiality Outside the Model for LLM Agents and Agent Swarms

ANONYMOUS AUTHOR(S)

Submission under double-anonymous review

Tool-using language-model agents can be redirected by untrusted content and may expose information through subsequent model requests. Because instructions, retrieved content, and tool results share the model context, prompt-level defenses alone cannot provide a deterministic enforcement boundary. We present Agent Security Policy (ASP), a declarative runtime policy model that moves authorization and information-flow decisions outside the model, as Content Security Policy moved script restrictions outside the web page. ASP combines a tool-path gateway for authorization, delegated scope, integrity tracking, and swarm-level containment with a model-API egress monitor that enforces confidentiality labels collected through tool interactions. The monitors share an engine-issued agent identity, so policy decisions can account for both an agent's authority and the data its context has accessed. We specify security invariants, implement a reference prototype, and evaluate it through designed attack scenarios, a counterfactual replay of 1,233,716 events from a research dataset of 46 agents, end-to-end tests, latency measurements, and deployment-confinement probes. In a designed indirect-injection scenario, capability withdrawal prevents a payment by an otherwise authorized agent that an allowlist alone permits. In the trace replay, the selected least-privilege policy would have blocked 7.9% of events, and task-scoped confidentiality labels marked 2.1% of model requests, compared with 9.8% under per-agent labels; this is a difference in how many requests carry a label, not a measured reduction in leakage. Task-scoped integrity tracking flagged 43.8% of shared-state writes, compared with 79.1% under per-agent tracking, while still preventing an agent from shedding taint by starting a fresh context. End to end, an agent that read a secret through the tool path was refused on the model path, and in a confined container deployment all 16 bypass probes failed. The in-process gateway decision took 0.047 ms at the median, and resolving an agent's identity added 1.33 ms per model request. These results characterize policy behavior and prototype feasibility rather than measured reductions in real-world harm. ASP's guarantees depend on complete mediation, trusted task lifecycle management, and deployment confinement; we discuss these assumptions and the remaining challenges in provenance tracking, deployment integration, and evaluation.

CCS Concepts: • Security and privacy → Access control; Information flow control; Software and application security; • Computing methodologies → Multi-agent systems.

Additional Key Words and Phrases: LLM agents, prompt injection, reference monitor, tool-use policy, information flow control, data loss prevention, multi-agent systems, Model Context Protocol, runtime enforcement

1 INTRODUCTION

Cross-site scripting was understood for years before browsers had a structural answer to it. The standard advice was to sanitize input, validate origins and audit code. That advice was right, and it put the whole defensive burden on the application, which was the thing under attack. Content Security Policy (CSP) changed where the decision was made [Stamm et al. 2010]. The server declares in a response header which resources a page may load, and the browser engine, which page scripts cannot reach, checks every fetch against that declaration. A fully compromised script cannot switch its own policy off because no interface for doing so exists in its process.

Tool-using agents built on large language models (LLMs) are in the position web applications were in before CSP. An agent's instructions live in its system prompt, which shares a context window with user messages, tool results and retrieved documents. Indirect prompt injection, where adversarial instructions arrive inside content

the agent reads rather than from the user, works against deployed systems [Greshake et al. 2023], and benchmarks show that current models and prompt-level defenses still follow injected instructions at meaningful rates [Debenedetti et al. 2024; Liu et al. 2024; Zhan et al. 2024]. When a retrieved file says to disregard previous directives, the model that reads it is the same model asked to uphold those directives. It settles the conflict by inference, and nothing outside it checks the result. The same context also leaves the system. Every turn sends the context window to a model provider, so a credential the agent read a minute ago, or customer records it was asked to summarize, travel with the next request whether or not anyone asked for them.

Multi-agent systems make this worse in two ways. Delegation creates confused deputies [Hardy 1988]: a sub-agent acting on an orchestrator's authority can be steered by content the orchestrator never saw. Messages between agents also give injected text a path from one context to the next [Lee and Tiwari 2024; Cohen et al. 2024]. Swarms also fail without any attacker, through hand-off loops, retry storms and write contention that no single agent's log reveals [Cemri et al. 2025; Hammond et al. 2025].

This article takes the position that the decisions about what an agent may do, and about what its context may carry out, should be moved out of the model, as CSP moved them out of the page. We make that position concrete and test it. Our contributions are:

- **A policy model.** Agent Security Policy (ASP) is a declarative, default-deny policy format covering tool scope, argument constraints, network destinations, content trust, delegation, swarm limits and confidentiality classes, with a report-only mode for adoption (Section 4).
- **A decision procedure with stated invariants.** We give a fixed evaluation order for every in-band action and show that it preserves two invariants: delegated scope never widens, and a context that has read untrusted content cannot regain a withdrawn capability. Task-scoped taint keeps the second invariant without tainting everything an agent later does (Sections 4.3 and 4.4).
- **Two monitors that share one identity.** A reference monitor on the tool path with three integration paths, including a Model Context Protocol (MCP) proxy in which the engine issues agent credentials, binds each credential to one task, reads tool semantics only from policy and dispatches tools itself; and an egress monitor on the model-API path that resolves each agent's credential to its identity, task and confidentiality labels, fails closed, and reports to the same recorder. A container profile makes the proxy an agent's only route to tool side effects (Section 5).
- **An evaluation on designed attacks and on real agent traces.** A layer ablation of an indirect-injection scenario, a counterfactual replay of 1,233,716 recorded events from a research dataset of 46 heterogeneous agents, a threshold sensitivity study, a measurement of how the unit of taint changes what integrity tracking can tell an operator, latency on four enforcement paths, an end-to-end run of both monitors with a confidentiality counterfactual on the real traces, and bypass probes against a confined and an unconfined deployment (Section 6).

No single mechanism here is new: allowlists, delegation attenuation, taint, high-water-mark labels, revocation and confinement all have long histories. The contribution is their integration around one engine-issued identity, so that what an agent did on the tool path decides what it may send on the model path, and a revocation on either path stops both. Table 13 in Section 9 compares this combination with prior systems.

Five results stand out. An allowlist, which is what most tool-calling frameworks offer, did not stop the injected payment in our scenario: an agent that was legitimately allowed to pay received the injected instruction by message and paid. Withdrawing capabilities from contaminated contexts was the only layer that stopped it. On real agent traces, a least-privilege policy would have flagged 7.9% of actions, most of them network reads outside a deliberately narrow allowlist, and 4,681 reads of credential stores by 22 agents. When we tracked taint per agent instead of per tool session, 79.1% of shared-state writes became contaminated, which leaves little signal for an operator; scoping taint to tasks that only a trusted controller can open brought this back to 43.8% without letting agents launder taint. On the model path, labels collected by the gateway let the egress monitor hold requests from contexts that had touched a credential store: 2.1% of model requests in the replay carried a credentials label with task-scoped labels, against 9.8% per agent, which measures how selective the label is, not how much data leaked; in a live run with both monitors an agent that read a secret was refused, as were its child agents and, after revocation, its requests. And the guarantee depends on deployment: in a container whose only route out is the gateway, every bypass probe failed, while the same agent with ordinary network access could reach the internet directly.

ASP is a proposal and our implementation is a research prototype. The gateway is a code boundary inside one process, not an operating-system boundary, and the protection it gives depends on agents having no route to tools other than through it. That confinement is the deployer's job; we give one profile that achieves it and test it, but it remains a precondition, not something the gateway provides. We state these limits where they apply and collect them in Section 8.

2 BACKGROUND

2.1 Tool-Using Agents and Agent Swarms

An LLM agent alternates between model inference and actions [Yao et al. 2023; Schick et al. 2023]. The model receives a context assembled from a system prompt, the conversation, and the results of earlier actions, and emits either text or a structured request to call a tool. A runtime executes the call and appends the result to the context. Tools range from read-only search to code execution, payments and writes to shared repositories. MCP standardizes how agents discover and invoke tools hosted by separate servers [Anthropic 2024].

Multi-agent frameworks compose such agents [Wu et al. 2023]. An orchestrator may spawn sub-agents with narrower tasks, agents may exchange messages, and several agents may write to the same resources. We use *swarm* for any such group whose behavior emerges from the interaction of several agents rather than from one plan.

2.2 Prompt Injection

Direct prompt injection places adversarial instructions in the user's own input [Perez and Ribeiro 2022]. Indirect injection places them in data the agent retrieves, such as web pages, email or documents [Greshake et al. 2023]. Liu et al. [2024] formalize both and show that a combined attack succeeds across many models and tasks. Agent benchmarks measure the same weakness when tools are involved: InjecAgent [Zhan et al. 2024] and AgentDojo [Debenedetti et al. 2024] report that injected tool outputs redirect agents into harmful calls. In multi-agent settings, injected text can replicate through messages and retrieval, as shown by Prompt Infection [Lee and Tiwari 2024],

self-replicating prompts in retrieval-augmented email assistants [Cohen et al. 2024] and infectious jailbreaks among multimodal agents [Gu et al. 2024]. Industry guidance ranks prompt injection first among risks to LLM applications and recommends mitigations that operate inside the application [OWASP GenAI Security Project 2025].

2.3 Content Security Policy as a Precedent

CSP has three parts that matter here [Stamm et al. 2010; West et al. 2016]. Policy is declared by the server in a header, outside the page. Enforcement happens in the browser engine, below the scripts it governs. Violations are sent to a reporting endpoint the page does not control, and a report-only mode lets operators observe what a policy would block before enforcing it. CSP also shows how such controls fail: Weichselbaum et al. [2016] found that most deployed allowlist policies could be bypassed, which led to stricter, nonce-based policies. We take two lessons from this history. Moving enforcement below the application works. Allowlists alone are weaker than they look.

2.4 Reference Monitors, Integrity and Confused Deputies

Anderson [1972] defined a reference monitor as a mechanism that mediates every access, cannot be tampered with, and is small enough to verify. Saltzer and Schroeder [1975] added least privilege and complete mediation to the vocabulary of protection. Schneider [2000] characterized what a monitor that watches an execution can enforce: safety properties, which a finite prefix of the execution can violate. A gateway that sees each agent action before it runs is such an execution monitor, and it inherits that limit, a point we return to in Section 4.7.

Biba's integrity model [Biba 1977] is the dual of the Bell and LaPadula confidentiality model [Bell and LaPadula 1973]. Under its low-water-mark policy, a subject that reads lower-integrity data takes on that lower integrity and loses the right to write to higher-integrity objects. Information flow control generalizes these ideas to labeled data [Denning 1976; Myers and Liskov 1997], and dynamic taint tracking applies them at run time [Enck et al. 2010]. Hardy's confused deputy [Hardy 1988] names the case where a program with legitimate authority is tricked into using it for another party. Capability systems answer it by passing authority explicitly and attenuating it on delegation [Dennis and Van Horn 1966; Miller et al. 2003; Birgisson et al. 2014].

2.5 Confidentiality and the Model-API Path

Data-loss prevention products inspect content leaving a network or an endpoint and match it against patterns, keywords and fingerprints of known documents. For an agent, the unit that leaves is the model request, and it carries the whole working context: system prompt, conversation, and the results of every tool call so far. Pattern matching on that text sees what the request looks like, not where its parts came from. A credential that was reformatted, summarized or split across turns no longer matches, while the fact that the context read a credential store does not change. Bell and LaPadula's confidentiality model [Bell and LaPadula 1973] supplies the provenance view: a subject that reads at a level is labelled with it, and its label only rises. We use that high-water mark to label agent contexts by what their tool calls touched, and let the monitor on the model path decide on the label.

3 THREAT MODEL AND SECURITY GOALS

3.1 System Model

A deployment has *agents*, which are LLM-driven processes; *tools*, which perform side effects; *model providers*, which serve the agents' inference requests; an *orchestrator* or registry that admits agents; a *gateway* that sits between agents and tools; and an *egress monitor* that sits between agents and model providers. An agent proposes *actions* of three kinds: a tool call with arguments, the spawn of a child agent with a requested scope, or a message to other agents. Content that enters an agent's context carries a trust level assigned by policy according to its source: the system prompt is high, user input medium, tool responses low and retrieved documents untrusted.

We treat agents as untrusted. Their reasoning can be steered by anything in their context, so what an agent says about its own actions is a claim, not evidence. The gateway, its policy file and the operator who writes the policy are trusted. Tools are trusted to do what their policy declaration says they do; Section 5.2 discusses how the MCP proxy limits the damage when a tool's declared semantics are wrong.

3.2 Adversaries and Failure Modes

- **A1: Indirect prompt injection.** The adversary controls content that an agent reads (a document, web page, email or tool output) and wants the agent, or any agent it talks to, to perform an action the operator did not intend.
- **A2: A compromised or misaligned agent.** An agent tries to exceed its delegated authority, for example by calling tools outside its scope, spawning a better-privileged helper, or recruiting other agents.
- **A3: Emergent failure without an adversary.** Recursive delegation, feedback loops between agents, retry storms and coordinated writes to shared state that degrade or damage the system although no single action is malicious.
- **A4: Data leaving through model requests.** Confidential data an agent has read reaches a model provider in a later request, either because the context carries it along or because injected text asks the agent to repeat it.

We do not consider an adversary who controls the gateway host, the policy file or the operator's credentials, nor attacks on the model's weights or training data. Data typed into browser chats, sent from other devices or exfiltrated through channels that are neither tools nor model requests is out of scope, and neither monitor intercepts TLS.

3.3 Security Goals

- **G1: Complete mediation of in-band actions.** Every tool call, spawn and message an agent sends through the gateway is decided before it takes effect.
- **G2: Least privilege with monotone delegation.** An agent can use only tools its policy and its ancestors' delegated scopes allow, and no spawn can widen a scope.
- **G3: Capability withdrawal after untrusted reads.** A context that has read untrusted content loses the capabilities the policy marks as sensitive, regardless of the agent's delegated scope.
- **G4: Swarm-level containment.** Patterns that only exist across agents (spread of taint, reciprocal loops, write contention, fan-out) are detected and can revoke the agents involved and their descendants.

- **G5: Observability.** Every decision is recorded with the rule that made it, the policy version and the origin of any taint, and a report-only mode produces the same records without blocking.
- **G6: Confidentiality on the model path.** A model request from a context that holds a data class is decided by that class's policy for the request's destination, using an identity and labels the agent cannot forge; a request whose identity cannot be verified fails closed.

G1 and G6 hold only for actions and requests that pass through the respective monitor. Whether an agent has another route to side effects, such as a shell or direct network access, is a property of the deployment, Section 5.4 states the confinement condition under which the MCP path closes that gap, and Section 6.8 tests a deployment that meets it. Table 12 in Section 8 separates the assumptions these goals rest on from the invariants we argue and the behaviors we tested.

4 THE AGENT SECURITY POLICY MODEL

4.1 Policy Language

An ASP policy is a JSON document validated against a strict schema when it is loaded; unknown fields are rejected, so a typo cannot silently disable a rule. Table 1 lists its sections. The `network`, `tools`, `content-trust`, `delegation` and `reporting` sections follow the original proposal; `swarm`, `inspection`, `mcp` and `confidentiality` are extensions this work adds. The policy never appears in a model's context. Like a CSP header, it is read by the enforcement layer, and the model cannot inspect or modify it.

Table 1. Sections of an ASP policy document

Section	Main fields	Purpose
`network`	`default`, `allow` (host patterns)	Destinations a tool call may reach; default deny
`tools`	`default`, `allow`, `deny`, `deny-arguments`	Tool allowlist and deny list (glob patterns); regular-expression constraints on arguments per tool
`content-trust`	trust per source, `contaminated-write`, `contaminated-deny`, `propagate-via-messages`	Trust levels; what a contaminated context loses; whether taint follows messages
`delegation`	`sub-agents: inherit-and-restrict`, `max-depth`	Scope attenuation on spawn and a depth cap
`reporting`	`violations`	Where structured violation records go
`swarm`	`window-seconds`, `step-budget`, `write-cap`, `repeated-intent`, `tripwires`	Rate limits per agent and resource; tripwire thresholds over a time window
`inspection`	`arguments`, `results`, detector families	Optional pattern detectors for secrets, personal data and injection markers
`mcp`	`servers`, per-tool `target`, `write`, `reads`, `resource`	Downstream MCP servers and the semantics of each exposed tool
`confidentiality`	`classes` (tool globs, argument patterns), `propagate-via-messages`	Data classes a context collects from its tool calls; read by the egress monitor

Listing 1 shows the core of the policy used for the synthetic scenarios in Section 6.2. It allows six mock tools, withdraws payment tools from contaminated contexts, propagates taint through messages, restricts sub-agents to subsets of their parent's scope, and sets a ten-second window for budgets and tripwires.

Listing 1. Core of the policy for the synthetic scenarios (abridged)

```
{ "asp-version": "1.0",
  "network": { "default": "deny", "allow": [] },
  "tools": { "default": "deny",
    "allow": ["mock:search.read", "mock:fs.read", "mock:ledger.read",
              "mock:payments.transfer", "mock:mcp:wiki.write",
              "mock:mcp:board.write"] },
  "content-trust": { "retrieved-document": "untrusted",
    "contaminated-write": "deny", "contaminated-deny": ["mock:payments.*"],
    "propagate-via-messages": true },
  "delegation": { "sub-agents": "inherit-and-restrict", "max-depth": 4 },
  "swarm": { "window-seconds": 10, "step-budget": 20, "write-cap": 6,
    "tripwires": { "taint-agents": 3, "delegation-loop": true } } }
```

4.2 State and Events

The gateway keeps a small amount of state. A registry maps each admitted agent to its parent, its depth and the span in which it was spawned. A scope map records the tool patterns delegated to each agent at spawn. Two taint maps record contamination: one maps a span (one tool session or one request context) to the identifier of the untrusted read that contaminated it, the other maps an agent to such an identifier. Under task scope (Section 4.4), the agent map is replaced by a map from (agent, task) pairs, and the gateway also records which tasks a trusted controller has opened for each agent and the task in which each child was spawned. A revocation set lists agents removed by the Sentinel, and sliding windows hold recent writes per resource, recent intents and recent attempts per agent.

An event e carries the acting agent a_e, its declared parent and parent span, an action kind, a target (a tool name or another agent), arguments, an optional network destination, the trust level of what it reads, whether it writes, the shared resource it writes, the agents it mentions and, for a spawn, the requested scope. Events also have a channel. In-band events pass through the gateway. Out-of-band events are activity the system observes but cannot stop, such as an edit made directly in a shared document; the gateway records them as *observed* and the Sentinel still analyzes them.

Let $\mathrm{anc}(a)$ be the chain of a and its admitted ancestors, and $\mathrm{S}(b)$ the scope delegated to b when it was spawned (absent for agents admitted by the registry). The tools an agent may call are those matching the policy allowlist A, not matching the deny list D, and matching every scope on its chain:

$$\mathrm{Eff}(a) \;=\; (A \setminus D) \;\cap\; \bigcap_{b \,\in\, \mathrm{anc}(a),\, \mathrm{S}(b)\ \text{defined}} \mathrm{S}(b) \tag{1}$$

A spawn that is allowed records $\mathrm{S}(c)$ for the child c. Taint origin is defined by looking up the span first and then walking the agent chain:

$$\mathrm{origin}(e) \;=\; \begin{cases} \sigma\big(\mathrm{span}(e)\big) & \text{if } \mathrm{span}(e) \in \mathrm{dom}\,\sigma \\ \tau(b\,*) & \text{if } b\,* \text{ is the first } b \in \mathrm{anc}(a_e) \text{ with } b \in \mathrm{dom}\,\tau \\ \bot & \text{otherwise} \end{cases} \tag{2}$$

where σ is the contaminated-span map and τ the contaminated-agent map. Under task scope, τ is keyed by (agent, task), and the walk up the chain uses, for each ancestor, the task in which its child was spawned. A context is clean when $\mathrm{origin}(e) = \bot$.

4.3 Decision Procedure

Algorithm 1 is the gateway's decision function. It returns one of *allow*, *drop*, *throttle* or *observed* together with the rule that decided it, and it evaluates checks in a fixed order. Identity and lineage come first because no other rule means anything for an actor the gateway does not know. Revocation and the step budget come next because they apply to every action kind. Spawns are then checked for duplicate identities and for scope widening. Tool calls go through the explicit deny list before the default-deny allowlist, so an explicit denial always wins. Delegated scope, argument constraints, content inspection and network scope follow, then capability withdrawal and the write rule for contaminated contexts, and finally write caps and repeated-intent throttling. The original proposal's three-step check (deny list, allowlist, network destination) corresponds to lines 13, 14 and 19.

ALGORITHM 1: ASP gateway decision for an in-band event e

```
Input: event e, policy P, gateway state (registry R, scopes S, taint maps, revoked set V)
Output: (decision, rule)
 1  if e.channel = out-of-band then return (observed, monitor.out_of_band)
 2  if e.agent not in R then return (drop, lineage.unknown)
 3  if declared parent or parent span of e differs from R[e.agent] then return (drop, lineage.invalid)
 4  if task scope and e.task was not opened for e.agent by a controller then return (drop, lineage.task_unknown)
 5  if depth(e.agent) + [e is a spawn] > P.delegation.max_depth then return (drop, lineage.max_depth)
 6  if e.agent or an ancestor is in V then return (drop, tripwire.revoked)
 7  if attempts by e.agent in window, including this one, > P.swarm.step_budget then
 8      return (throttle, budget.steps)              // blocked attempts count too
 9  if e is a spawn and e.target in R then return (drop, lineage.duplicate)
10  if e is a spawn and some requested pattern is outside S(b) for some b on anc(e.agent) then
11      return (drop, delegation.scope_expansion)
12  if e is a tool call then
13      if e.target matches P.tools.deny then return (drop, tools.deny)
14      if P.tools.default = deny and e.target does not match P.tools.allow then
15          return (drop, tools.default_deny)
16      if e.target is outside S(b) for some b on anc(e.agent) then return (drop, delegation.out_of_scope)
17      if an argument rule for e.target matches e.arguments then return (drop, tools.deny_arguments)
18      if inspection is set to deny and a detector fires on e.arguments then return (drop, inspection.*)
19      if e.network is not allowed by P.network then return (drop, network.default_deny)
20      o <- origin(e)
21      if o != none and e.target matches P.content_trust.contaminated_deny then
22          return (drop, trust.contaminated_tool)
23      if o != none and e writes and P.content_trust.contaminated_write = deny then
24          return (drop, trust.contaminated_write)
25      if e writes and writes to e.resource in window >= P.swarm.write_cap then return (drop, tools.write_cap)
26      if the same normalized intent and target came from >= min_agents agents, limit reached then
27          return (throttle, semantic.identical_intent)
28  return (allow, policy.allow)
```

After a decision, the gateway updates state. An allowed spawn registers the child with its parent, depth, span and requested scope. If the event executed (it was allowed, or the gateway runs in report-only mode) and it read

untrusted content while its context was clean, the event's span is marked contaminated with the event's identifier as origin. If propagation is enabled and a message from a contaminated context executed, each recipient that is not yet tainted is tainted with the same origin. Every decision, including allows, is appended to the flight recorder with the rule, the reason, the policy version, the mode and the taint origin.

Two properties follow directly from the procedure and the update rules. We state them for an enforcing gateway and for actions that pass through it; Table 12 lists the assumptions they rest on.

PROPOSITION 1 (SCOPE NON-EXPANSION). *For every agent c admitted by an allowed spawn from parent p, the scope recorded for c satisfies* $S(c) \subseteq S(p)$*, and every tool call allowed for an agent a has a target in Eff(a).*

PROOF SKETCH. *By induction on the sequence of decisions. Initially only the root and registry-admitted agents exist and no scopes are recorded. A spawn is allowed only if line 10 finds every requested pattern inside the scope of every agent on the spawner's chain, which includes the spawner's own scope when it has one, so the recorded scope is a subset of each ancestor's. Scopes are written only at spawn and never changed afterwards. A tool call reaches line 28 only after lines 13 to 16 have checked the deny list, the allowlist and every scope on the chain, which is the definition of Eff(a) in Equation 1.*

PROPOSITION 2 (NO REGAINED CAPABILITY). *Once origin(e) is defined for the contexts of an agent, no later tool call from those contexts to a target matching the contaminated-deny list is allowed, in any mode where taint is attached to the agent or to a task that only a controller can open.*

PROOF SKETCH. *The taint maps only grow: entries are added on untrusted reads and on propagation and never removed during a session. Equation 2 consults the agent chain, so descendants spawned after contamination inherit the origin. Line 21 runs before any rule that can return allow for a tool call. Under task scope the same argument holds within a task, and line 4 refuses any task identifier a controller did not open, so the only way to a clean context is a new task opened by a trusted principal. Where taint is attached only to a span, an agent can obtain a clean context by starting a new span; Section 5 describes the integration paths that attach taint to the agent or the task for this reason, and Section 6.5 measures what each choice costs.*

The step budget at line 7 counts blocked attempts as well as allowed ones. An agent stuck in a retry loop therefore stays throttled until it backs off, instead of being released as soon as its denied attempts age out.

4.4 Content Trust and Capability Withdrawal

Under Biba's low-water-mark rule a subject that reads low-integrity data drops to that level [Biba 1977]. ASP applies the rule to capabilities rather than to labels on objects: a contaminated context loses the tools the policy lists under `contaminated-deny`, and if `contaminated-write` is `deny` it loses all writes. Withdrawal is independent of scope. That independence is the point. In a confused-deputy attack the deputy holds the capability legitimately, so a scope check passes; what has changed is that its instructions now descend from untrusted content. Each contaminated decision records the identifier of the read it descends from, so an operator can trace a blocked payment back to the document that caused it.

Propagation through messages is optional because its value depends on the topology. In a system where agents send directed messages to named peers, propagation follows the path an injected instruction would take. In a broadcast chat room where every agent reads every message, it marks nearly everyone. Section 6.5 quantifies this on real data.

Scope of taint. The policy chooses the unit that taint attaches to. Under the default, `actor`, an untrusted read taints the reading span and, on the mediated integration paths of Section 5, the agent. Agent-level taint is what makes Proposition 2 hold against an agent that invents a fresh span, but it is coarse: once an agent has read anything untrusted, everything it does later is contaminated. Under `task`, taint attaches to an (agent, task) pair, and only a trusted controller (the operator, a scaffold, or the engine when it issues a credential) can open a task. A fresh span inside the same task stays tainted; a task the controller opens later starts clean, and children spawned in a tainted task inherit its taint. Opening a new task is therefore an explicit declassification decision made by a principal the agent cannot impersonate. How much this buys depends on whether task boundaries are meaningful in the deployment, which Section 6.5 measures.

4.5 Confidentiality Labels

Taint tracks integrity: what may influence a context. Confidentiality labels track the opposite direction: what a context holds and may therefore leak. A policy's `confidentiality` section declares data classes, each defined by tool globs and by regular expressions over a call's canonical arguments. An executed tool call that matches a class adds it to the acting unit, the same unit taint uses: the agent under the default scope, or the (agent, task) pair under task scope. Labels follow Bell and LaPadula's high-water mark: they only accumulate, children spawned from a labelled context inherit them, and when the policy enables it they follow messages. Every recorder row carries the labels its context held after the action.

The gateway does not act on labels itself, because the data has already been read when a label is added; withdrawing tools at that point is what capability withdrawal does. Labels matter on the other path. An egress monitor that receives a model request asks the engine for the requesting agent's identity, current task and labels, and treats each label as a source label of its own data classes. Source labels are the first, deterministic step of its decision, so a class whose action for the request's destination is *block* refuses the request regardless of what its text contains.

PROPOSITION 3 (LABELLED CONTEXTS DO NOT REACH BLOCKING DESTINATIONS). *For every model request routed through an egress monitor that requires engine credentials, if the requesting context holds class c and c's enforced action for the request's destination is block, the request is refused.*

PROOF SKETCH. *A request without a credential is refused, and so is one whose credential the engine does not resolve. A resolved credential yields the labels of the agent's current task and of every ancestor's task at spawn time; since labels are never removed, a context that held c when it read the data still holds it. The monitor evaluates source labels for every class before any classifier, takes the strictest verdict across matched classes, and a classifier can only make the verdict stricter. So c's block applies. The argument covers only requests that pass through the monitor, and a label records that a context touched the class, not that a particular request contains it.*

4.6 Swarm Tripwires and Revocation

The Sentinel maintains a weighted directed graph of executed interactions, with one edge per ordered pair of agents or agent and resource, whose weight counts repetitions. A second graph holds only message edges inside the current window. Six tripwires run over these structures, each with thresholds from the policy:

- *Reciprocal consensus*: a strongly connected cluster of at least k_c agents in the windowed message graph whose directed density reaches θ_d (Equation 3).

- *Shared-state loop*: at least k_s distinct agents writing the same resource within the window.
- *Fan-out*: an agent that has spawned more admitted children than the policy allows.
- *Echo cascade*: at least k_e agents posting messages whose token sets overlap above a Jaccard threshold within the window.
- *Injection spread*: one untrusted origin reaching at least k_t agents through propagated taint.
- *Delegation loop*: a chain of hand-offs, followed through messages, that returns to an agent already on the chain.

$$\mathrm{dens}(C) = \frac{|E_C|}{|C|(|C|-1)} \geq \theta_d \tag{3}$$

where E_C is the set of distinct ordered pairs among the cluster's members that exchanged a message in the window. Density counts distinct pairs, not message volume, so a single noisy pair cannot raise it. When a tripwire fires and feedback is enabled, the Sentinel adds the implicated agents to the revocation set and increments the policy version; line 6 of Algorithm 1 then refuses their actions and those of their descendants. An alert that overlaps a recent one of the same kind by at least half of its agents is treated as the same episode, so a cluster that gains or loses one member does not fire again.

4.7 Report-Only Mode and What a Monitor Can Enforce

In report-only mode the gateway evaluates and records every action but blocks nothing, and each record states what the policy *would* have done. Budgets and rate windows count only actions that would have been allowed, so a report-only run predicts what an enforcing run would have permitted. This mode serves two purposes. It lets operators measure false positives on their own workloads before enforcing, as CSP's report-only header did. It also makes historical traces analyzable: the actions already happened, and the decisions describe a counterfactual.

ASP inherits the limits of execution monitoring [Schneider 2000; Ligatti et al. 2005]. It can stop a prefix of an execution that violates a safety property: a forbidden tool call, a widening spawn, a write after an untrusted read. It cannot enforce properties that depend on what an agent will do later, and it cannot undo an action that has already run. Tripwires partly address this, since they act on patterns over several past actions, but a tripwire fires after the pattern exists, and in our implementation revocation stops the next call rather than one already dispatched.

5 REFERENCE IMPLEMENTATION

We implemented the tool-path monitor, ASP-RM, in Python, with a FastAPI service, a web dashboard and a typed API specification, and the model-path monitor, ASP-EG, as a single Go binary. Figure 1 shows how they fit together. Both decision cores are small: the gateway, the policy schema and the Sentinel together are 649 lines of Python, and the egress monitor's decision path, including its detectors and the engine link, is 1,427 lines of Go. That keeps Anderson's third property, verifiability, within reach of a reader.

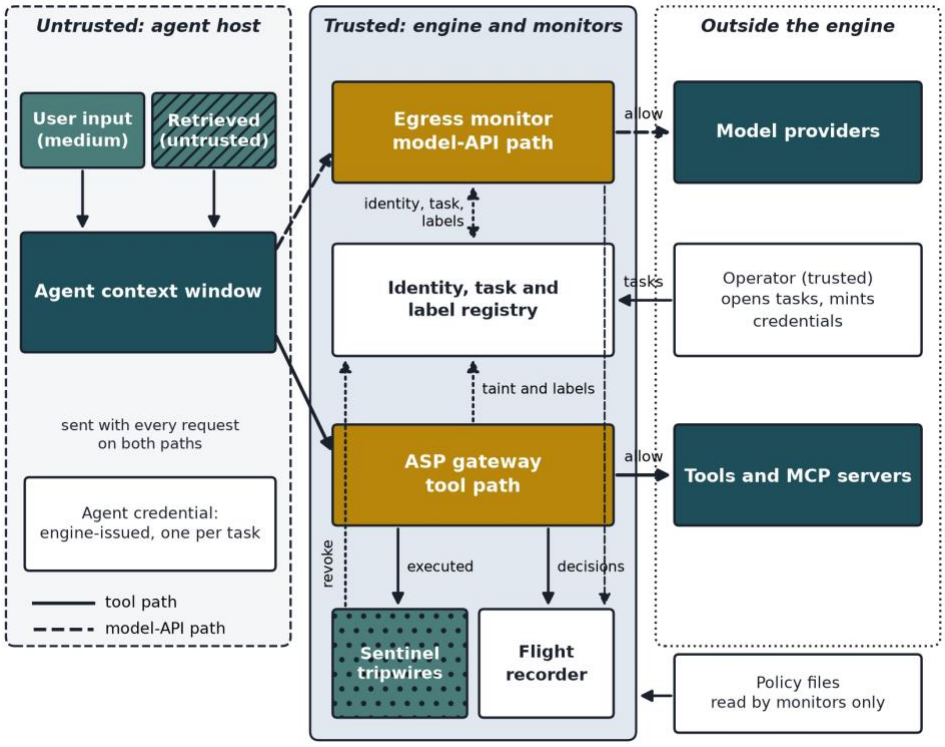


Fig. 1. Trust zones, identity and the two monitor paths. The agent host (left) is untrusted; the engine and both monitors (center) are trusted; providers, tools and the operator are outside the engine. Tool calls, spawns and messages take the tool path (solid) to the ASP gateway; model requests take the model-API path (dashed) to the egress monitor. Every request carries the agent's engine-issued credential, one per task. The registry holds identity, task, taint and labels: the gateway writes taint and labels, the egress monitor resolves them, the Sentinel's revocations reach both paths, and only the operator opens tasks. Both monitors record every decision in the flight recorder.

5.1 Gateway, Sentinel and Flight Recorder

The gateway implements Algorithm 1 over a normalized event model shared by all data sources: synthetic scenarios, live sessions, MCP calls and historical replays. Tool patterns use shell-style globs, so `mock:payments.*` covers every payment tool. Argument constraints are regular expressions over a canonical serialization of the arguments. The Sentinel uses NetworkX for strongly connected components and density. The flight recorder stores, for every event, the decision, rule, reason, policy version, mode, structured violation records, contamination flag, taint origin and, for replayed data, a reference to the source row. For calls the engine executes itself, it also stores the observed outcome (succeeded, failed or not started); an *allow* records permission, and only the engine's observation records that a tool ran.

5.2 Three Integration Paths

How strongly the gateway mediates depends on how agents reach tools. ASP-RM offers three paths, and we keep their guarantees distinct.

Cooperative guard. A dependency-free Python library wraps an agent's tools. Before each call it asks the gateway, in process or over authenticated HTTP, and raises an exception on refusal. This path is easy to adopt in existing frameworks but cooperative: code that calls a tool directly bypasses it. Taint on this path is attached to spans.

Engine-owned execution. For a small set of registered tools, the agent sends only the tool name and arguments to the engine. The engine verifies the caller's session and lineage, derives the tool's metadata (target, trust of what it reads, whether it writes, which resource) from its own registry rather than from the request, evaluates the policy and dispatches the tool under the same lock. Skipping the client library gains nothing, because the tool body lives in the engine. Untrusted reads taint the agent, not only the span, so a fresh span cannot launder contamination. Each request carries an idempotency key; an exact retry returns the original outcome without a second evaluation or dispatch, and a changed payload under the same key is refused.

MCP proxy. The engine exposes an MCP endpoint. When an agent is spawned through the gateway, the engine issues it a credential bound to its identity and lineage and stores only a hash. Downstream MCP servers are declared in the policy's `mcp` section and launched by the engine over standard input and output, with a per-session workspace and the MCP SDK's minimal environment rather than the engine's. The policy, not the server, defines each tool's semantics: its gateway target, whether it writes, which resource it touches, whether its output is untrusted, and which argument holds a network destination. Undeclared tools are neither listed nor callable. Each credential is one task: under task-scoped taint, an agent that read untrusted content stays tainted for as long as it uses that credential, and only the session owner can rotate it, which retires the old credential and opens a new, clean task. Rotation is journaled, so a durable session restores it after a restart. A refused call returns an error result naming the rule, and the downstream server never sees it. Results from tools marked untrusted taint the calling agent, and optional result inspection can redact or block secrets and personal data or taint the reader when injection markers appear.

5.3 The Model-API Path

ASP-EG is a reverse proxy for model APIs that listens on loopback only. Agents reach it by overriding their provider base URL, for example `ANTHROPIC_BASE_URL`, with the upstream host as the first path segment; the proxy opens its own TLS connection upstream and does no interception. It decides each request with a pipeline that evaluates deterministic signals first: source labels, fingerprints of known documents, then patterns and keywords (credential formats, card numbers validated with the Luhn check, personal data and organization-defined markers). An optional classifier is consulted only for classes the deterministic steps left undecided and that could still make the verdict stricter, and by default it receives features of the request rather than its text. Verdicts are allow, translate, hold and block, decided per data class and destination category, and each class runs in report or enforce mode. The request body is held in memory for the decision only; decisions record class identifiers, rule names, verdict and destination, never content.

The engine link makes the two monitors one system. An agent's model requests carry its engine-issued MCP credential in a header. The monitor resolves it with a loopback call authenticated by a shared secret and receives the agent's identity, its current task, its confidentiality labels, whether it is tainted and whether it is revoked. The labels become source labels, the agent identity is recorded as engine-issued rather than declared, and a revoked agent's model requests are refused, so a branch the Sentinel revokes loses its model access along with its tools. The

monitor fails closed: a credential the engine does not recognize, an unreachable engine, or, when the deployment requires it, a missing credential refuses the request. The credential header is removed before forwarding and never logged. After each decision about an engine-identified agent, the monitor posts a content-free record to the engine, which stores it as an observed, out-of-band row whose provenance names the egress monitor; delivery is asynchronous through a bounded queue, so a slow recorder delays no request. In our implementation, hold and translate act as block, because the approval flow and the surrogate translator they need are not built.

5.4 Confinement Condition

The MCP proxy removes the agent's cooperation from enforcement only if the proxy is the agent's sole route to side effects. That is a deployment property: the agent host must expose no shell or native tools outside the proxy, must have no network route to downstream servers, and must not hold their credentials, and the agent's MCP configuration must be set by the operator rather than chosen by the model. The engine does not enforce these conditions. Our adapters for agent frameworks help by removing built-in tools from the agent's configuration, but an operating-system boundary is still the deployer's job. We provide one container profile that meets the condition. The agent runs in its own container attached only to an internal network whose only other host is the engine, so it has no default route and no external name resolution. Downstream MCP servers run inside the engine container over standard input and output, with no network listener, and the engine's own credentials exist only there. The agent receives its own MCP credential through a read-only volume and nothing else; the operator's token stays in a volume the agent does not mount. The agent's root file system is read-only, all Linux capabilities are dropped and privilege escalation is disabled. Section 6.8 tests this profile against a compromised agent. It is one profile among many possible ones, and any deployment that changes these properties has to re-establish the condition.

Task-scoped taint and labels add a second deployment condition: the controller must manage task lifecycles correctly, because opening a task is declassification. We operate tasks under four rules. Only an operator opens tasks; in our implementation, rotation requires the operator or administrator role, and an agent's MCP credential cannot authenticate to that interface at all (Section 6.8, probes A2 and A3). A task opens at a boundary set by a principal outside the agent, such as a new assignment from a person or a scheduler, and never because an agent asks for one or is blocked by a withdrawn capability. Before opening a new task for an agent whose current task is tainted or labelled, the operator reviews what that task read, using the taint origin and labels on its recorder rows; children spawned in the old task keep its taint and labels whatever the operator decides. Long-running tasks accumulate taint and labels, so work is split at natural boundaries, and the share of tainted and labelled tasks is watched during the report-only phase (Section 7.4). A controller that ignores these rules recreates the laundering problem that task scope closes.

5.5 Supporting Mechanisms

Several mechanisms make the prototype usable as a service, and they are not part of the policy model. Sessions can be journaled to SQLite and restored by exact replay after a restart; any state change made outside the gateway's decision function, such as taint pinned by the MCP proxy, is journaled explicitly so that replay reproduces it. Access to sessions is scoped to tenants derived from verified identity-provider tokens, with viewer, operator and

administrator roles. Content inspection uses pattern detectors for credentials, personal data and common injection phrasings; findings record detector names and counts, never the matched text.

5.6 Verification

The engine has 230 automated tests in 15 suites (Table 2), and the egress monitor has 113 test cases, five of them for the engine link. One suite pins the decisions, alerts and revocations of the four synthetic scenarios so that later changes cannot alter them silently. The MCP suite drives a real MCP client over HTTP into the engine, which launches a sample workspace server, and reaches every check in Algorithm 1 through that path. For each refusal it confirms three things: the client receives an error naming the rule, the recorder marks the call as not started, and the server's own audit log has no entry for it.

Table 2. Automated test suites of the reference implementation

Suite	Tests	What it covers
Engine	40	Policy evaluation order, detectors, normalization, synthetic regression of all four scenarios
Task-scoped taint	13	Laundering through fresh spans refused within a task; controller-opened tasks start clean; unopened task identifiers refused; inheritance by children; rotation over MCP and across a restart
Egress link	9	Confidentiality labels from tools and arguments, inheritance and propagation, task scope, egress endpoint authentication, recorder rows and durability
Execution	27	Engine-owned dispatch, metadata forgery, taint laundering through fresh spans, locking, idempotent retries
Journal	36	Client-side journal for uncertain outcomes and recovery
MCP gateway	17	Every Algorithm 1 check through a real MCP client and server; refused calls never reach the server
MCP proxy	17	Credential issue and binding, undeclared tools, lineage, end-to-end proxy behavior
Content inspection	13	Argument and result detectors, redaction, blocking, injection-marker taint
Access	12	Token verification, foreign sessions, spoofed identity headers
Completion	12	Execution receipts for caller-run tools
Persistence	8	Journaled sessions restored by exact replay
Real agents	8	Offline model fixtures through the authenticated execution routes
Access roles	7	Tenant isolation and role limits
Missing completion	6	Calls whose outcome was never reported
Adapters	5	Agent framework connection helpers (skipped without the optional SDK environment)

All 230 engine tests pass in our environment, with five skipped when the optional framework SDKs are not installed, and all 113 egress monitor cases pass. The monitor's engine-link cases check that engine labels block a request, that a clean agent is forwarded without its credential header, that unknown, unreachable and revoked identities fail closed, that a required credential is enforced, and that recorder events carry no content or credential.

6 EVALUATION

We address seven questions.

- **RQ1.** Does the gateway contain the attacks and failures in the threat model, and which layer is responsible?
- **RQ2.** What would a least-privilege ASP policy flag on real, heterogeneous agents doing open-ended work?

- **RQ3.** How sensitive are those results to policy thresholds?
- **RQ4.** What does the choice of taint granularity do to the signal that integrity tracking provides?
- **RQ5.** What does enforcement cost on each integration path?
- **RQ6.** Do confidentiality labels let the model path hold what the tool path exposed, and at what cost?
- **RQ7.** Does a confined deployment close the routes around the gateway, and what opens without confinement?

6.1 Setup

All measurements ran on one Apple M2 laptop with 8 GB of memory, macOS 26.6.2 and Python 3.13.5. Synthetic scenarios are fixed event sequences replayed through the gateway in enforce mode. Real-data experiments replay a private, access-on-request research dataset in report-only mode. Latency experiments run the actual services. The end-to-end experiment runs the engine and the egress monitor binary as separate processes on the same machine. The confinement experiment runs the engine and the agent as separate containers under Docker Desktop 28.4. Scripts, policies and aggregate outputs for every table and figure are in the artifact (see Artifact Availability).

6.2 RQ1: Containment of Designed Attacks and Failures

Four synthetic scenarios exercise the threat model. *Normal* is a baseline of three agents with distinct tasks (15 events). *Swarm attack* covers A2 and A3: recursive spawning, reciprocal messaging and coordinated writes to a shared wiki (41 events). *Injection* follows the confused-deputy story from the original proposal (14 events). An orchestrator spawns a receipt parser with read-only scope, a ledger agent with payment rights and a notifier with board-write rights. The parser reads an invoice containing the instruction to transfer $5,000, then tries to pay, tries to spawn a helper with payment rights, and messages the ledger agent and the notifier. *Runaway* has no adversary (33 events): a planner, executor and critic hand work around in a cycle, and a scraper retries the same fetch 22 times. Table 3 gives the decisions with tripwire feedback on and off.

Table 3. Decisions in the synthetic scenarios (enforce mode). Feedback on means tripwire alerts revoke the agents involved.

Scenario	Feedback	Allow	Drop	Throttle	Observed	Alerts	Revoked
Normal	on / off	12 / 12	0 / 0	0 / 0	3 / 3	0 / 0	0 / 0
Swarm attack	on / off	18 / 21	11 / 4	2 / 6	10 / 10	3 / 3	7 / 0
Injection	on / off	9 / 10	5 / 4	0 / 0	0 / 0	1 / 1	3 / 0
Runaway	on / off	29 / 31	2 / 0	2 / 2	0 / 0	2 / 2	3 / 0

In the normal run nothing is blocked and no tripwire fires. In the swarm attack, one spawn exceeds the depth cap, two writes hit the shared-resource cap and two calls are throttled as repeated intent; three tripwires fire (shared-state loop, reciprocal consensus and fan-out), and with feedback the eight later actions of revoked agents are dropped. In the runaway scenario the delegation-loop tripwire fires when the critic hands work back to the planner, and the step budget throttles the scraper's 21st and 22nd attempts with or without feedback.

Figure 2 traces the injection scenario. The parser's own payment is dropped because payments are outside its delegated scope, and its attempt to spawn a payment-capable helper is dropped as scope widening. The ledger agent holds payment rights, so its scope check passes, but it received the parser's message and inherits the parser's taint;

its payment is dropped by capability withdrawal. When the taint reaches the third agent, the injection-spread tripwire fires and revokes all three, and the clean orchestrator's status post is allowed.

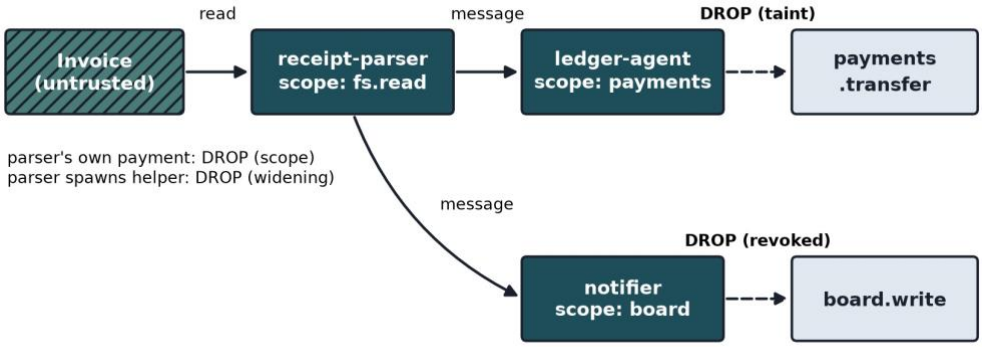


Fig. 2. Taint flow in the injection scenario. The receipt parser reads an untrusted invoice. Its own payment and its attempt to spawn a payment-capable helper are refused by delegation rules. Messages carry its taint to the ledger agent, whose payment is refused by capability withdrawal, and to the notifier, whose write is refused after the injection-spread tripwire revokes all three agents.

To see which layer does what, we removed layers one at a time and counted payments that would execute (Table 4). Removing tripwire feedback changes nothing about payments, because the notifier's write is then refused by the contaminated-write rule instead of by revocation. Removing taint lets the ledger agent pay. Removing delegated scopes lets the parser spawn its helper, but taint still stops both payments. With neither taint nor scopes, both the parser and the ledger agent pay; that configuration is exactly the deny-list, allowlist and network check of the original three-step algorithm.

Table 4. Layer ablation of the injection scenario: payments that would execute

Configuration	Parser pays	Ledger agent pays	Helper spawned	Payments executed
Full ASP	no (scope)	no (taint)	no	0
Without tripwire feedback	no (scope)	no (taint)	no	0
Without taint	no (scope)	yes	no	1
Without delegated scopes	no (taint)	no (taint)	yes	0
Without taint or scopes	yes	yes	yes	2
Allowlist only (deny list, allowlist, network)	yes	yes	yes	2

The ablation has a plain reading. Allowlists and scopes limit which agent may use which tool. They cannot tell a legitimate payment from one an agent was talked into, because in both cases the authorized agent makes the call. Only a rule that conditions authority on where the agent's instructions came from separates the two.

6.3 RQ2: Counterfactual Replay of Real Agent Traces

Data. We used the AI Village dataset [AI Digest 2026], an ongoing record of LLM agents from several providers living and working in a shared virtual environment on open-ended goals, with shared chat rooms, computer-use

sessions with a shell, and a coding-agent scaffold. The dataset is private; its maintainers grant access to researchers on request, and they also offered it to participants of a hackathon on agent swarm dynamics [AI Village and Grove Research 2026]. We used the export of 20 September 2026, covering 389 days. Our adapter normalizes chat messages into message events whose recipients are the agents they mention, computer-use shell commands and tool actions into tool events, and coding-agent tool calls into tool events with their real tool names. Shell commands are classified for network destination, untrusted reads (web fetches and retrieved files) and shared-state writes (pushes to shared repositories and mutating calls to their APIs). Human chat participants are collapsed into one anonymous actor with no text retained. The result is 1,233,716 events: 1,050,231 tool events and 183,485 chat messages. The 46 agent identities produced 1,223,724 of them; the remaining 9,992 are chat messages from the single anonymous actor that stands for human participants and the scaffold, which the gateway records as out-of-band observations.

Policy. We wrote a least-privilege starting policy for this environment before looking at results (Appendix A). It allows the village's tools and the coding agent's standard tools, allows network access to the code-hosting, office-suite and organizer domains the agents were asked to use, and denies everything else. Five argument rules deny piping a downloaded script to a shell, recursive deletion of the root or home directory, force pushes to shared history, reads of credential stores and printing environment secrets. Writes after untrusted reads are reported but not blocked, and taint does not propagate (Section 6.5 explains why). Tripwires use a ten-minute window. Each day is replayed separately in report-only mode with feedback off, using the agents admitted by that day as the registry.

Results. Table 5 and Figure 3 summarize the replay. The policy would have dropped or throttled 97,472 actions, 7.9% of all events. Most are network destinations outside the allowlist (86,206, or 34.9% of the 246,783 tool actions with a network destination). On inspection these are ordinary research reads of news sites, preprint servers and prediction-market interfaces. They show how far a strict default-deny network scope sits from what open-ended agents actually do, which is the case for report-only rollout in Section 7.

Table 5. AI Village replay in report-only mode: what the policy would have done over 389 days. Events are normalized actions: 1,050,231 tool actions and 183,485 chat messages, 1,233,716 in all; 1,223,724 by 46 agent identities and 9,992 out-of-band messages from the anonymous human and scaffold actor

Signal	Count	Denominator	Share
Would drop or throttle	97,472	1,233,716 events	7.9%
Network destination outside allowlist	86,206	246,783 tool actions with a network destination	34.9%
Argument constraints	5,512	1,050,231 tool actions	0.52%
Credential-store reads (22 of 46 agents; 98.2% from 24 March 2026)	4,681	1,050,231 tool actions	0.45%
Force pushes to shared history (26 of 46 agents)	439	1,050,231 tool actions	0.04%
Environment secret disclosure (29 of 46 agents)	384	1,050,231 tool actions	0.04%
Remote script piped to a shell (5 of 46 agents)	8	1,050,231 tool actions	<0.01%
Write cap per shared resource	4,010	101,332 shared-state writes	4.0%
Repeated identical intent	1,744	1,233,716 events	0.14%
Shared-state writes after untrusted reads (reported, not blocked)	44,430	101,332 shared-state writes	43.8%
Tripwire alerts (876 consensus, 113 shared-state, 7 echo)	996	389 days	2.6 per day
Days without any alert	160	389 days	41.1%
Per-agent would-block rate: median, range 0.10% to 39.0%	4.9%	each agent's events; 46 agents	median

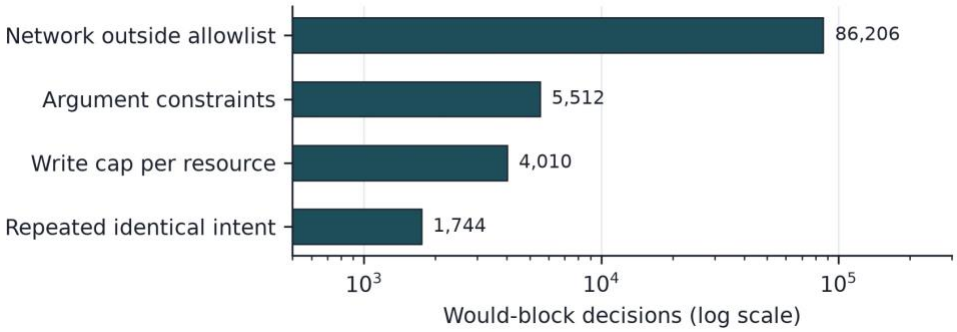


Fig. 3. Would-block decisions by rule in the AI Village replay (log scale). Network scope dominates; argument constraints, write caps and repeated-intent throttling account for the rest.

The argument constraints are the more interesting findings for security. Agents read credential stores 4,681 times, mostly to extract the code-hosting token from a command-line client's configuration and call the hosting API directly. In this environment that was routine, and agents usually did it to work around a limitation of the provided tool. It is also the kind of standing access that a least-privilege deployment would route through a scoped tool, and an agent that can read the token can send it anywhere its network scope allows. Agents printed environment secrets 384 times and force-pushed to shared history 439 times.

The dataset's change log matters when reading these numbers. Two scaffold changes alter the data's shape: an automatic nudging mechanism introduced on 10 February 2026 and persistent computer use from 24 March 2026. Table 6 splits the results at those dates. The would-block rate is 6.4% before the first change, 1.9% between the two and 9.2% after the second, and 98.2% of credential-store reads fall in the last period. These differences reflect changes in the scaffolding as much as in agent behavior, so we do not interpret them as trends.

Table 6. AI Village replay by scaffold period (rate = would-block events as a share of the period's events)

Period	Days	Events	Would-block events	Rate	Credential-store reads	Alerts
Before 10 Feb 2026	229	170,813	10,865	6.4%	73	343
10 Feb to 23 Mar 2026	30	147,809	2,846	1.9%	10	178
From 24 Mar 2026	130	915,094	83,761	9.2%	4,598	475

All 996 tripwire alerts are of the consensus, shared-state and echo kinds; the dataset has no sub-agent spawns, so depth, fan-out and scope rules are not exercised, and taint propagation and the delegation-loop tripwire are off. Many alerts reflect collaboration the agents were asked to do, which is why alerts in this setting are signals for review rather than verdicts. A replay records what a policy would have said about actions that already happened. It does not show that ASP would have prevented any outcome, and the dataset contains no labels saying which actions were harmful.

6.4 RQ3: Sensitivity to Thresholds

We varied one threshold at a time and replayed the full dataset again (Table 7). Each change moves only the rule it governs; network and argument decisions are identical across variants because they do not depend on thresholds. Halving the write cap from 12 to 6 writes per resource per window triples write-cap decisions (4,010 to 12,888); doubling it to 24 cuts them by 80% (803). The repeated-intent limit behaves similarly but less steeply. Tripwire alerts are most sensitive to the consensus density threshold: lowering it from 0.6 to 0.5 more than doubles alerts and leaves only 133 quiet days, while raising it to 0.8 cuts alerts to 282.

Table 7. Threshold sensitivity on the AI Village replay (baseline in the middle row of each group). Decisions by the varied rule and total would-block events are counts of events out of 1,233,716; days are out of 389

Parameter	Value	Decisions by this rule	Total would-block events	Alerts	Days without alert
Write cap per resource	6	12,888	106,348	996	160
	12	4,010	97,472	996	160
	24	803	94,265	996	160
Repeated-intent limit	3	2,353	98,079	996	160
	5	1,744	97,472	996	160
	8	909	96,637	996	160
Consensus density	0.5	1,985	97,472	2,105	133
	0.6	876	97,472	996	160
	0.8	162	97,472	282	243

For the consensus rows, decisions by this rule are reciprocal-consensus alerts, not blocked events. The thresholds were set on this dataset and should be treated as starting points; Section 8 discusses why they will not transfer unchanged.

6.5 RQ4: Taint Granularity

Integrity tracking is only useful if a meaningful fraction of contexts stays clean. We measured how the unit of taint affects that fraction. In the replay, taint is attached to spans, which in this dataset correspond to individual computer-use or coding sessions. We compared five configurations on the same 101,332 shared-state writes: span-level taint as the replay implements it, agent-level taint in which an untrusted read taints the whole agent for the rest of the day (as the engine-owned and MCP paths do under the default scope), agent-level taint with propagation through chat messages, and task-scoped taint with and without propagation. For task scope, each computer-use or coding session is a task, because the village scaffold, not the agent, starts sessions; chat messages belong to the sender's current task.

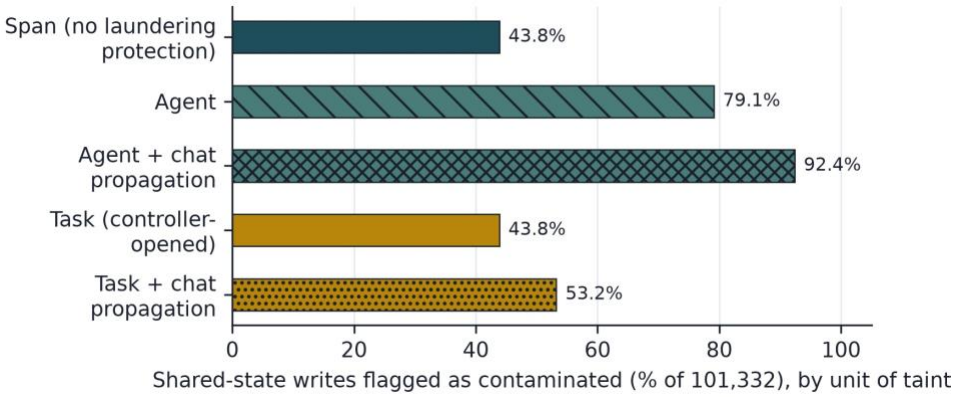


Fig. 4. Share of shared-state writes flagged as coming from a contaminated context, by unit of taint, on the AI Village replay. Task-scoped rows (solid and dotted bars at the bottom) refuse laundering through fresh spans; the span row does not.

Figure 4 shows the result. Span-level taint flags 43.8% of writes but offers no protection against an agent that starts a new span. Agent-level taint flags 79.1%, and a majority of active agents is tainted on 317 of 389 days, a median of 40 minutes after the day's first event (interquartile range 18 to 77 minutes). Adding propagation through chat flags 92.4%; a majority is tainted on 364 days, a median of 20 minutes into the day (10 to 43). With the span-level implementation, enabling propagation has no effect at all, because agents' chat messages are separate spans from the sessions in which they read untrusted content, so the taint never reaches a message. Task-scoped taint flags 43.8% of writes, the same as span-level taint, while refusing the laundering that span-level taint allows; 28,477 of the 60,590 tasks (47.0%) end tainted. With propagation through chat it flags 53.2% of writes, and 64.5% of tasks end tainted, against 92.4% of writes for agent-level taint with propagation.

The village is an extreme case: its agents read the open web as part of their job, and they share broadcast rooms. Still, the measurement makes a design point concrete. Agent-level taint closes the laundering gap in Proposition 2 but saturates quickly in such workloads, and once most contexts are tainted, withdrawing capabilities from tainted contexts amounts to withdrawing them from everyone. Task scope keeps both properties here because the scaffold's sessions are natural task boundaries. A deployment whose tasks run for days, or whose controller opens new tasks carelessly, would lose one property or the other; Section 9 relates this to systems that label data rather than contexts.

6.6 RQ5: Enforcement Cost

We measured per-call latency on four paths, each in three runs (Figure 5 and Table 8). The guard workload has 20 agents making 5,000 decisions, four tool calls for every message, with event timestamps 100 ms apart so that agents stay inside their step budget; tripwire feedback is off, but detection still runs, so the cost includes the Sentinel's graph update. The HTTP guard runs against the real service over loopback, with an RS256-signed session token verified on every request. The MCP workload sends 1,000 calls through a real MCP client: 900 allowed file reads and 100 calls to a deny-listed tool. The baseline sends the same 1,000 reads to the same sample MCP server directly over standard input and output, without the engine. We report the median across runs of each percentile.

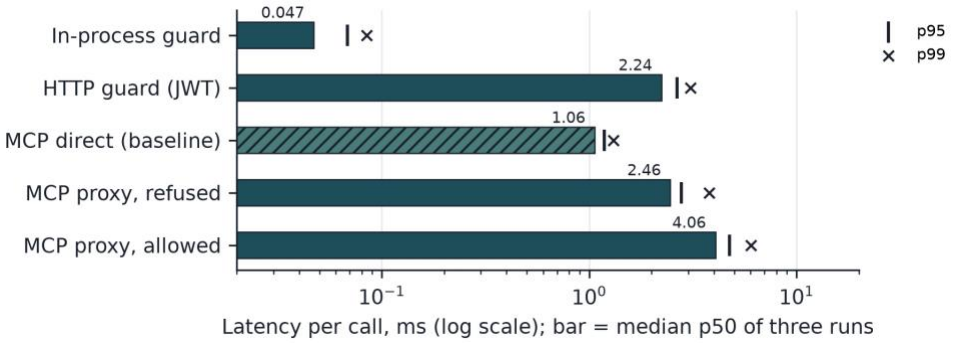


Fig. 5. Latency per call on each enforcement path (log scale). Bars show the median p50 over three runs; tick and cross markers show p95 and p99. The hatched bar is the direct MCP baseline without the gateway.

Table 8. Latency per call in milliseconds (median of three runs)

Path	Calls per run	p50	p95	p99
In-process guard (Algorithm 1 and Sentinel)	5,000	0.047	0.068	0.084
HTTP guard, loopback, token verified per request	5,000	2.24	2.65	3.07
Direct MCP call, no gateway (baseline)	1,000	1.06	1.19	1.31
MCP proxy, allowed call	900	4.06	4.77	6.02
MCP proxy, refused call	100	2.46	2.78	3.78

The policy decision itself is cheap: 47 microseconds at the median, including graph maintenance. Transport and authentication dominate the other paths. Most of the HTTP guard's 2.24 ms is token verification and request handling, and the MCP proxy adds 3.00 ms at the median over a direct call for allowed calls. Refused calls through the proxy are faster than allowed ones because the engine never contacts the downstream server. Across three runs, p50 values varied by at most 4%. The numbers were measured on the final code, including task-scoped taint; an earlier measurement without it gave p50 values within 5% of these. The recorder confirmed that all 100 refused calls in each run were decided by the deny-list rule and none reached the server.

6.7 RQ6: Confidentiality on the Model Path

End to end. We ran the engine and the egress monitor as separate processes. The engine served one MCP session under task-scoped taint with a confidentiality class `credentials` defined by a path pattern. The egress monitor ran in enforce mode with one class, `credentials`, identified only by the engine label and blocked for every destination, and it required engine credentials. Model requests were addressed to a reserved host name that never resolves, so a forwarded request ends in the monitor's upstream error and no traffic left the machine. Table 9 lists the steps.

Table 9. End-to-end run of both monitors (forwarded means the egress monitor allowed the request and attempted the upstream call)

Step	Action	Expected	Observed
E1	Clean agent sends a model request	forwarded	forwarded
E2	Agent reads a secret through the MCP proxy	label credentials	label credentials

Step	Action	Expected	Observed
E3	Same agent sends its next model request	refused	refused
E4	Model request without a credential	refused	refused
E5	Model request with a forged credential	refused	refused
E6	Child spawned from the labelled task sends a model request	refused	refused
E7	Owner opens a new task for the agent; request with the new credential	forwarded	forwarded
E8	Request with the retired credential of the labelled task	refused	refused
E9	Engine revokes the agent; its next model request	refused	refused
E10	Engine recorder holds one row per engine-identified decision	5 rows	5 rows, 3 refused

All ten steps behaved as designed. The secret was read through the tool path, where the gateway allowed it; the model path refused what followed. The child agent was refused although it never read the secret, because it was spawned in the labelled task. Opening a new task restored model access for the agent, which is the controller's declassification step, while the retired credential stayed refused. Revocation by the engine cut off the agent's model access. Only the five decisions about engine-identified agents reached the recorder; requests without a verifiable identity have nothing to attach to and stay in the monitor's own log.

Counterfactual on real traces. We replayed the AI Village data again with a confidentiality class `credentials` defined by the replay policy's own argument rules for credential-store reads and secret printing. Each in-band action is one model turn, and we count a turn's request as labelled if its context held the label before the turn, which is the context that was sent. Table 10 gives the results for agent-level and task-scoped labels, with and without propagation through chat.

Table 10. Model requests in the AI Village replay that would have carried a credentials label. A model request is one in-band tool action or chat message by an agent (1,223,724 in all: the 1,233,716 replay events less 9,992 out-of-band messages from the anonymous human and scaffold actor, which are not agent model requests); an agent-day is one agent active on one day (4,797); a task is one scaffold session (60,590)

Label unit	Labelled model requests	Share of 1,223,724	Labelled units
Agent	119,545	9.8%	351 of 4,797 agent-days
Task (controller-opened)	25,788	2.1%	1,310 of 60,590 tasks
Agent, with chat propagation	515,552	42.1%	2,047 of 4,797 agent-days
Task, with chat propagation	64,988	5.3%	5,393 of 60,590 tasks

With task-scoped labels, 25,788 model requests on 165 days came from contexts that had touched a credential store; an egress monitor deciding on labels would have held them. The difference between 2.1% and 9.8% is a difference in how many requests carry a label. It is not a measured reduction in data leakage: the dataset does not record which requests contained a secret, and neither figure counts leaks. Task scope is also only as safe as its task boundaries. If a controller opens a clean task while a secret persists outside the session, in an agent's memory notes, a file it wrote or a message it sent, requests in the new task carry the secret without the label, and the monitor allows them. The village agents keep memory notes across sessions, so the 2.1% may undercount requests whose context held a secret. Per-agent labels would have held 9.8% of all requests, and propagation through broadcast chat would have held 42.1%, which, as with taint, is too coarse to be useful. The numbers measure the cost of letting secrets into a context, not a cost the monitors must pay. Most of the 4,681 credential-store reads extracted a token

so that the agent could call an API itself. A tool that performed the call with a scoped token would have kept the token out of the context, and none of those requests would have been labelled. The labels show where that refactoring is needed.

Cost and detectors. Resolving the credential added 1.33 ms per model request at the median: 1.96 ms with resolution against 0.63 ms with the same labels supplied in a header, over 1,000 refused requests each, measured at the client. The monitor's own decision takes 29 microseconds at the median on its labelled corpus. Its pattern and keyword detectors classified that corpus of 20 synthetic samples without a miss or a false positive, which is far too small to estimate accuracy. None of the results in this section depend on them: every refusal in Table 9 and every labelled request in Table 10 comes from engine labels.

6.8 RQ7: Confinement and Bypass

The MCP proxy's guarantee rests on the confinement condition of Section 5.4, so we tested a deployment that meets it. We ran the container profile from Section 5.4 with the engine serving one session under task-scoped taint and one agent minted with scope over files, mail, a web fetch tool and a shell tool family whose execute tool is deny-listed. The agent container held a probe that acts as a compromised agent: it has arbitrary code execution inside its container and its own engine-issued credential, and it tries to reach a side effect or more authority without an allowed gateway decision. A control call first checks that sanctioned tool calls still succeed. We ran the same probe image twice: confined, and unconfined, where the agent is also attached to an ordinary network with internet access. Table 11 lists the probes and outcomes.

Table 11. Bypass probes by a compromised agent with code execution in its container (closed means the route failed)

Probe	Attempt	Confined	Unconfined
N1	TCP connection to a public IP address	closed	open
N2	Resolve a public host name	closed	open
N3	HTTP request to a public web site	closed	open
N4	Find other services on the engine host (ports 1-1024, 8000-8100)	closed	closed
A1	Create a session on the owner API without a token	closed	closed
A2	Rotate its own task (self-declassify)	closed	closed
A3	Use its MCP credential as an owner token to mint a helper	closed	closed
A4	Present a forged owner token	closed	closed
A5	Call the MCP endpoint with a forged agent credential	closed	closed
T1	Call a tool the policy does not declare	closed	closed
T2	Call a deny-listed tool that its scope pattern covers	closed	closed
I1	After an untrusted read, send mail (withdrawn capability)	closed	closed
I2	After an untrusted read, write shared files	closed	closed
I3	Open a fresh MCP connection to obtain a clean context	closed	closed
S1	Read the engine's downstream credential from its environment	closed	closed
S2	Read the operator's token file	closed	closed

In the confined profile all 16 routes were closed and the control calls succeeded. Network egress failed at the network layer: there was no route to a public address and no name resolution. The only open port on the engine host was the engine's own. The owner API refused the agent in every form we tried, including a self-rotation that would have declassified its own task. The gateway refused the undeclared and deny-listed tools and, after the

untrusted read, the withdrawn capabilities, including on a fresh connection. In the unconfined profile, the gateway-level probes behaved identically, but the agent could open connections to the internet, resolve names and fetch web pages. Those three routes bypass the gateway entirely: an agent that can reach a model provider or an API directly needs no tool call to act.

The experiment supports a narrow claim. The gateway's rules held regardless of deployment, and only confinement closed the routes around it, which is the division of responsibility Section 5.4 states. The probe checks the routes it enumerates; it does not show that no other route exists, and it does not test container escapes, kernel vulnerabilities or an operator who mounts more into the agent container. The probe agent also had no model behind it, which is why every route could be closed. A deployed agent needs one outbound route, to its model provider, and in our design that route goes through the egress monitor. We tested the two monitors together on one host (Section 6.7) but not inside this container profile, because the egress monitor is built as a per-workstation proxy that listens only on loopback.

7 DISCUSSION

7.1 Allowlists Are Necessary and Not Sufficient

The ablation in Table 4 is small, but its logic is general. Any check that depends only on which agent calls which tool will pass a confused deputy, because the deputy is authorized. CSP's history points the same way: allowlists that looked strict were bypassable in most deployments until policies started tying permission to provenance through nonces and hashes [Weichselbaum et al. 2016]. For agents, provenance means the source of the instructions an agent is acting on. Integrity labels are a coarse but deterministic way to approximate it, and they do not depend on the model recognizing an attack.

7.2 Two Paths, One Identity

Neither monitor could do the other's job. The gateway never sees model requests. The egress monitor cannot see which tools an agent used or what it read through them, and matching patterns in outbound text finds a secret only while it is still recognizable in the request. Sharing the engine-issued identity turns the gateway's knowledge into the egress monitor's first, deterministic signal, and turns the egress monitor's decisions into rows in the same recorder, so one timeline shows a read on the tool path and the refused request that followed. It also extends containment: a branch the Sentinel revokes loses its model access along with its tools, which matters because an agent that can still call its model can still plan. The link costs one loopback call per model request. Its most useful output may be diagnostic. In the replay, the labels pointed at the places where secrets entered contexts, and the remedy there is to broker credentials through scoped tools, not to hold more requests.

7.3 The Unit of Taint Is a Design Decision

Section 6.5 shows a tension. Attaching taint to the agent prevents laundering through a fresh context, which Proposition 2 needs. On a workload where most agents read untrusted content, it also taints most of the system within an hour. Confidentiality labels share the unit and the tension, as Table 10 shows. Task scope resolves the tension when tasks have meaningful boundaries: it was as selective as span-level taint on the village data and still refused laundering. It moves a decision to the controller, though. Opening a new task declassifies, so the controller

must open tasks at points where the agent's prior context genuinely no longer matters, such as a new assignment from a human, and not merely because an agent is blocked. A controller that rotates tasks on request recreates the laundering problem one level up. Two other levers remain. Taint can be attached to data rather than to contexts, as information-flow planners do [Costa et al. 2025; Debenedetti et al. 2025], at the price of controlling how the agent's program is structured. And the policy can declare which tools are sensitive enough to deserve withdrawal, which ASP already does with `contaminated-deny`, so that saturation costs little where it does not matter. Our results argue for measuring saturation on a deployment's own traffic, in report-only mode, before choosing a scope.

7.4 Report-Only First

A default-deny network policy, written in good faith before looking at the data, would have blocked about a third of the village agents' network actions. Most of them were ordinary work. That is the strongest argument for the adoption path CSP took: deploy in report-only mode, read the violation reports, adjust the policy, then enforce rule by rule. ASP's report-only mode produces exactly the records an enforcing gateway would produce, which makes this loop possible without changing the deployment. We recommend treating report-only operation as a required phase rather than an option. Run the policy in report-only mode on real traffic for long enough to cover the deployment's routine work. Review would-block decisions by rule with the owners of the affected tools and data, and separate violations from legitimate work the policy did not anticipate. Adjust allowlists and thresholds, and record the false-positive rate per rule. Then enforce rule by rule, starting with rules whose would-block decisions were all violations, such as credential-store reads, and keep report-only coverage for rules still being tuned.

7.5 Where Mediation Holds

The three integration paths in Section 5 make different claims, and conflating them would overstate the result. The cooperative guard is a convenience and a source of telemetry, not a security boundary. Engine-owned execution and the MCP proxy make the gateway the only code that can run a tool, and the MCP proxy extends that property to any MCP server. Neither prevents an agent with a shell from acting on its own. The CSP analogy holds here too: CSP protects nothing a page can reach through a browser extension or a separate process. For agents, the corresponding requirement is that the deployment confine the agent's runtime, so that the gateway is its only route to effects. Section 6.8 makes this concrete: with confinement every probe failed, and without it the same agent reached the internet directly while every gateway rule still held. Operators should treat confinement as part of the policy's trusted computing base and verify it per deployment, for example by running a probe like ours as part of release testing.

7.6 Relation to Standardization

ASP is a proposal, not an adopted standard, and nothing in this article claims conformance to one. We see value in three pieces becoming shared conventions regardless of who defines them: a machine-readable policy format that sits beside an agent's deployment rather than in its prompt, a structured violation record that security tooling can ingest, and a report-only mode. Risk frameworks ask organizations to document controls and monitor deployed AI systems [National Institute of Standards and Technology 2023], and records of this kind are one way to supply that evidence. Policy languages for authorization already exist [OASIS 2013; Cutler et al. 2024]; what agents add is the need to condition decisions on lineage, provenance of instructions and swarm-level state.

8 LIMITATIONS AND THREATS TO VALIDITY

Table 12 separates three kinds of statement this article makes. Assumptions are conditions the guarantees need and that we did not test. Invariants follow from the decision procedure under those assumptions, by the arguments in Section 4 and the unit tests in Section 5.6. Tested behaviors were observed in the specific settings listed and hold only there.

Table 12. Assumptions, invariants and tested behaviors

Kind	Statement	Basis and scope
Assumption	Complete mediation: an agent's only routes to tools and to model providers are the gateway and the egress monitor	Deployment condition (Section 5.4); tested for the tool path in one container profile (Table 11), not for the model path inside containers
Assumption	The engine, its policy files, its host and the operator are trusted and uncompromised	Outside the threat model (Section 3.2); not tested
Assumption	Tasks are opened only at boundaries set outside the agent, under the rules of Section 5.4	Operational condition; enforced only in that the owner API requires the operator role; operator practice not evaluated
Assumption	Tool semantics declared in the policy (writes, untrusted output, network argument) are correct, and sources are labelled correctly	Policy author's responsibility; a mislabelled tool is judged by its label
Assumption	Each agent's credential stays with that agent	Not tested; the egress monitor never logs or forwards it
Invariant	Delegated scope never widens; every allowed tool call is within the agent's effective scope (Proposition 1)	Proof sketch (Section 4.3); MCP gateway and engine test suites
Invariant	A tainted context cannot regain a withdrawn capability under agent or task scope (Proposition 2)	Proof sketch (Section 4.3); task-scoped taint suite, including laundering through fresh spans
Invariant	A labelled context's model request is refused where its class blocks the destination (Proposition 3)	Proof sketch (Section 4.5); egress link suites in both monitors
Tested	Injection scenario contained; capability withdrawal is the only layer that stops the authorized agent's payment	Synthetic scenarios, enforce mode (Tables 3 and 4)
Tested	Policy behavior on real traces: would-block decisions, taint and label rates, threshold sensitivity	One dataset, report-only, counterfactual (Tables 5 to 7 and 10, Figure 4)
Tested	Both monitors together: labelled, unverified, inherited and revoked requests refused; declassified task allowed	One host, ten steps (Table 9)
Tested	Bypass routes closed in a confined deployment and open without it	One container profile, 16 enumerated probes (Table 11)
Tested	Enforcement cost	One laptop, sample MCP server (Table 8, Section 6.7)

Not a tamper-proof boundary. The gateway, recorder and Sentinel are modules in one process. Code with access to the host can modify the policy, the engine or its state. Our tamper-resistance claim is limited to agents whose only route to tools is the engine, as Section 5.4 states. The confinement experiment tests one container profile against routes we enumerated; it does not establish that no other route exists, and it does not cover container escapes, kernel vulnerabilities, misconfigured deployments or the identity provider, which the lab replaced with a local signing key.

The model path. The egress monitor sees only requests routed through it. Browser chats, desktop applications with built-in models, and other devices are outside it, and it does not intercept TLS. Labels mark what a context touched, not what a request contains, so a labelled context's harmless request is held too; labels defined by argument patterns are only as good as the patterns. The monitor's pattern detectors and its optional classifier have

not been evaluated beyond a 20-sample synthetic corpus. Hold and translate act as block in our implementation. The two monitors were tested together on one host, and the egress monitor's loopback-only listener suits a workstation; forcing a containerized agent's model traffic through it was not tested.

Task boundaries. Task-scoped taint is only as good as the controller's task boundaries. In the replay, tasks were scaffold sessions, which suited the data. Deployments must decide what a task is and who may open one, following the rules in Section 5.4; we did not evaluate how operators follow them in practice, and a controller that opens tasks whenever an agent asks would undo the protection.

Internal validity. The synthetic scenarios were designed by the authors to exercise specific rules. They show that the mechanisms behave as specified, not how often such attacks occur or succeed. The ablation covers one attack structure. In the replay, untrusted reads and writes are inferred from shell commands and tool names by a classifier we wrote; it does not see graphical actions such as clicks and typing into documents, so shared-state writes made that way are missed.

Construct validity. A would-block decision is not harm. The dataset has no labels marking harmful actions, and many flagged actions were legitimate work. Untrusted content is assigned by source category, not detected; a document from an untrusted source may be benign, and an injection could arrive through a source labeled trusted. Taint therefore approximates the provenance of instructions; it does not measure it.

External validity. We evaluated one real dataset from one environment, with agents that mostly communicate in broadcast rooms and never spawn sub-agents. Delegation rules, depth limits, fan-out and inheritance of taint by children were exercised only in synthetic scenarios and unit tests, not on real multi-agent traces, because the dataset contains no sub-agent spawns. Thresholds were calibrated on this dataset. Scaffold changes during the dataset's lifetime affect cross-period comparisons, as Table 6 shows. Latency was measured on one laptop with a sample MCP server; production networks and real tools will add their own costs.

Heuristic detectors. Repeated-intent throttling compares normalized text, and echo detection uses token overlap; neither recognizes paraphrase. Content inspection is pattern-based and misses encoded, paraphrased or non-English injections. These detectors complement the deterministic rules; they do not replace them, and we did not measure their miss rates on adversarial inputs.

Identity. Session authentication establishes the owner of a session, not the identity of each agent in it, except in MCP proxy mode, where agents hold engine-issued credentials. Credentials there last as long as their session, and revocation stops the next call, not one already dispatched.

9 RELATED WORK

9.1 Model-Level Defenses

A line of work trains or prompts models to separate instructions from data. The instruction hierarchy trains models to give lower priority to instructions from lower-privileged sources [Wallace et al. 2024]. StruQ separates prompt and data channels and fine-tunes the model to follow only the prompt [Chen et al. 2025]. Spotlighting marks or encodes untrusted input so the model can recognize it [Hines et al. 2024], and activation-based detectors look for task drift caused by injected text [Abdelnabi et al. 2025]. Guard models such as Llama Guard classify inputs and outputs [Inan et al. 2023], and programmable guardrail frameworks route conversations through developer-

defined rails [Rebedea et al. 2023]. These defenses lower the probability that an injection succeeds. ASP is complementary: it limits what an agent can do when one succeeds, and it does not depend on the model's judgment to do so.

9.2 System-Level Designs for Agents

Closest to our work are designs that move security decisions out of the model. Willison's dual-LLM pattern keeps a privileged model away from untrusted text and passes references instead of content [Willison 2023]. CaMeL extracts control flow from the trusted user query, runs it in a custom interpreter that tracks provenance, and enforces capability policies when tools are called [Debenedetti et al. 2025]. FIDES attaches confidentiality and integrity labels to data in an agent planner and checks them before consequential actions [Costa et al. 2025]. Beurer-Kellner et al. [2025] catalog design patterns that constrain agents so untrusted input cannot trigger consequential actions. IsolateGPT isolates apps from each other in LLM-based systems with separate model instances and a mediated communication protocol [Wu et al. 2025]. Prompt Flow Integrity combines agent isolation, handling of untrusted data and guardrails against privilege escalation [Kim et al. 2025], and Progent provides a language for expressing fine-grained tool privileges for agents [Shi et al. 2025].

ASP shares the premise of these systems and differs in emphasis. Data-level labeling, as in CaMeL and FIDES, is more precise than our context-level taint and avoids the saturation we measured, but it requires control over how the agent's plan is executed. FIDES also tracks confidentiality and can selectively hide information from the planner [Costa et al. 2025]; ASP's confidentiality labels are coarser, attached to contexts, and enforced at the request to the model provider, which works with unmodified agents. ASP sits at the tool and model-API boundaries, works with unmodified agents, and adds four things we did not find combined elsewhere: confidentiality labels shared from the tool path with a monitor on the model path, attenuation of scope along delegation chains, swarm-level tripwires with revocation of descendants, and a report-only mode evaluated on a large corpus of real multi-agent traces.

Table 13. Mechanisms described by prior agent-security designs and by ASP. Y: described; P: partly, or for a narrower case; n.d.: not described in that work

System	Tool auth.	Delegation scope	Integrity taint	Model-request confid.	Swarm revocation	Confinement	Shared identity
Dual LLM [Willison 2023]	P	n.d.	P	n.d.	n.d.	n.d.	n.d.
CaMeL [Debenedetti et al. 2025]	Y	n.d.	Y	n.d.	n.d.	P	n.d.
FIDES [Costa et al. 2025]	Y	n.d.	Y	P	n.d.	n.d.	n.d.
IsolateGPT [Wu et al. 2025]	P	n.d.	n.d.	n.d.	n.d.	Y	n.d.
Prompt Flow Integrity [Kim et al. 2025]	Y	n.d.	P	n.d.	n.d.	P	n.d.
Progent [Shi et al. 2025]	Y	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.
Policy Cards [Mavračić 2025]	P	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.
Policies on Paths [Kaptein et al. 2026]	Y	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.
ASP (this work)	Y	Y	Y	Y	Y	P	Y

Table 13 records only what each work describes; n.d. does not mean a design is incompatible with the mechanism. ASP is marked P for confinement, which it states and tests in one profile but does not provide. FIDES is

marked P on model-request confidentiality because it hides labelled data from its own planner model rather than deciding on each request a context sends to a provider.

9.3 Runtime Governance and Policy Formats

Policy Cards propose a machine-readable, deployment-layer specification of an agent's operational and regulatory constraints, with allow and deny rules, obligations and evidence requirements, intended to connect to runtime enforcement and audit [Mavračić 2025]. Kaptein et al. [2026] model compliance policies as functions of an agent's identity, partial execution path, proposed action and organizational state, and argue that runtime evaluation is required for path-dependent policies. Our step budgets, write caps and tripwires are path-dependent in this sense. ASP differs from both mainly in where enforcement is anchored and in its security framing: the policy is read only by a monitor outside the model, and its rules target integrity of instructions rather than regulatory compliance. General authorization languages such as XACML [OASIS 2013] and Cedar [Cutler et al. 2024] could express parts of an ASP policy; lineage, taint and swarm state would need to be supplied to them as attributes.

9.4 Multi-Agent Risks

Hammond et al. [2025] survey risks that arise when many advanced agents interact, including miscoordination, conflict and collusion. Cemri et al. [2025] analyze failures in multi-agent LLM frameworks and find that many stem from system design and inter-agent misalignment rather than from single-model errors. Attacks that spread between agents [Lee and Tiwari 2024; Cohen et al. 2024; Gu et al. 2024] motivate our taint propagation and injection-spread tripwire. Our replay adds an empirical view of how often real agents in a shared environment trigger such patterns, and how quickly integrity labels spread when they share broadcast channels.

9.5 Classical Mechanisms

ASP applies established ideas to a new kind of principal. The gateway is a reference monitor [Anderson 1972] and an execution monitor in Schneider's sense [Schneider 2000; Erlingsson and Schneider 1999; Ligatti et al. 2005]. Tool-call interposition resembles system-call interposition [Goldberg et al. 1996; Provos 2003], and it inherits that approach's known pitfalls, such as gaps between what is checked and what is executed [Garfinkel 2003]; engine-owned dispatch addresses this by deriving tool metadata itself and evaluating and dispatching under one lock. Inherit-and-restrict delegation follows capability attenuation [Dennis and Van Horn 1966; Miller et al. 2003; Birgisson et al. 2014]. Capability withdrawal applies Biba's low-water mark [Biba 1977] with dynamic taint tracking [Denning 1976; Enck et al. 2010]. The deployment model follows zero-trust principles of explicit verification and least privilege [Rose et al. 2020], and the CSP precedent [Stamm et al. 2010; Barth et al. 2008] shaped the separation of declaration, enforcement and reporting.

10 CONCLUSION

Browsers became defensible against script injection when the decision about what a page may do moved into a layer the page could not reach. Agents need the same move. This article specified Agent Security Policy, a declarative policy model and a deterministic decision procedure for tool calls, delegation and messages, extended it with confidentiality labels for the path to model providers, implemented it as two small monitors that share an engine-

issued identity, and evaluated them on designed attacks, an end-to-end run and more than a million real agent actions. Allowlists alone did not stop a confused-deputy payment; capability withdrawal after untrusted reads did. Labels collected on the tool path let the model path refuse what followed a secret read, and in real traces showed that 2.1% of model requests came from task contexts that had touched a credential store, a measure of label selectivity rather than of leakage. A least-privilege policy flagged credential-store reads and secret disclosure in real agent traces, and also showed how much ordinary work a strict network scope would block, which is why report-only mode matters. The unit of taint decides whether integrity tracking stays informative: per-agent taint saturated, while taint scoped to controller-opened tasks stayed selective without letting agents launder it. And the guarantee is only as good as the deployment's confinement, which we tested and which remains the deployer's responsibility. Defining task boundaries well, and moving from context-level to data-level labels without giving up compatibility with unmodified agents, are the problems we would most like to see addressed next.

ARTIFACT AVAILABILITY

An anonymized archive with the reference implementation, every policy used for the results, the synthetic scenarios, the egress monitor with its engine link, the test suites, the container profile and bypass probe, the end-to-end script, and the scripts that produced every table and figure is provided to reviewers as a supplementary file with this submission (`asp-artifact.zip`); its README lists the commands that reproduce each table and figure. The AI Village data is not redistributed. It is available from its maintainers under their research terms [AI Digest 2026], and our scripts rebuild the normalized store and the aggregate results from an approved download.

USE OF GENERATIVE AI

Generative AI tools assisted in developing the prototype: the two monitors, their test suites, the confinement lab and the analysis scripts. The authors directed the design, reviewed the code, and ran every test and experiment; all reported numbers come from executing that code. Generative AI tools also assisted in producing the figures: Figures 1 to 5 are drawn by a plotting script written with that assistance, which renders the architecture diagram and plots the measured results; no figure is a generated image, and the script is in the artifact. The AI Village dataset analyzed in Section 6 records the behavior of LLM agents and is the object of study, not a writing aid [AI Digest 2026].

DATA ETHICS STATEMENT

This work analyzes the AI Village dataset, a private research dataset whose maintainers grant access on request [AI Digest 2026; AI Village and Grove Research 2026], under its stated terms, which permit research and analysis and prohibit re-identification. We report aggregates only. Human chat participants were collapsed into a single anonymous actor and their message text was never stored; chat text and shell commands were truncated and scrubbed for e-mail addresses, telephone numbers and token-shaped strings in addition to the dataset's own redaction. We did not use any credential appearing in the data. No new data was collected from human participants.

REFERENCES

Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. 2025. Get My Drift? Catching LLM Task Drift with Activation Deltas. In *Proceedings of the 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*.

AI Digest. 2026. AI Village Dataset. Retrieved 20 September 2026 from https://theaidigest.org/village

James P. Anderson. 1972. *Computer Security Technology Planning Study*. Technical Report ESD-TR-73-51, Vol. II. Electronic Systems Division, Air Force Systems Command, Hanscom AFB, MA.

Anthropic. 2024. Model Context Protocol Specification. https://modelcontextprotocol.io

Adam Barth, Collin Jackson, and Charles Reis. 2008. *The Security Architecture of the Chromium Browser*. Technical Report. Stanford University.

D. Elliott Bell and Leonard J. LaPadula. 1973. *Secure Computer Systems: Mathematical Foundations*. Technical Report MTR-2547, Vol. I. The MITRE Corporation, Bedford, MA.

Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. arXiv:2506.08837.

Kenneth J. Biba. 1977. *Integrity Considerations for Secure Computer Systems*. Technical Report MTR-3153, Rev. 1 (ESD-TR-76-372). The MITRE Corporation, Bedford, MA.

Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. 2014. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: Defending Against Prompt Injection with Structured Queries. In *Proceedings of the 34th USENIX Security Symposium*.

Stav Cohen, Ron Bitton, and Ben Nassi. 2024. Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications. arXiv:2403.02817.

Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643.

Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024). https://doi.org/10.1145/3649835

Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*.

Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236-243. https://doi.org/10.1145/360051.360056

Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (1966), 143-155. https://doi.org/10.1145/365230.365252

William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Úlfar Erlingsson and Fred B. Schneider. 1999. SASI Enforcement of Security Policies: A Retrospective. In *Proceedings of the 1999 Workshop on New Security Paradigms (NSPW)*. 87-95.

Tal Garfinkel. 2003. Traps and Pitfalls: Practical Problems in System Call Interposition Based Security Tools. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

Ian Goldberg, David Wagner, Randi Thomas, and Eric A. Brewer. 1996. A Secure Environment for Untrusted Helper Applications: Confining the Wily Hacker. In *Proceedings of the 6th USENIX Security Symposium*.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. 79-90. https://doi.org/10.1145/3605764.3623985

Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Lewis Hammond et al. 2025. *Multi-Agent Risks from Advanced AI*. Cooperative AI Foundation Technical Report 1. arXiv:2502.14143.

Norm Hardy. 1988. The Confused Deputy (or Why Capabilities Might Have Been Invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36-38. https://doi.org/10.1145/54289.871709

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kıcıman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674.

Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. 2026. Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586.

Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv:2503.15547.

Donghyun Lee and Mo Tiwari. 2024. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. arXiv:2410.07283.

Jay Ligatti, Lujo Bauer, and David Walker. 2005. Edit Automata: Enforcement Mechanisms for Run-time Security Policies. *International Journal of Information Security* 4, 1-2 (2005), 2-16.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *Proceedings of the 33rd USENIX Security Symposium*. 1831-1847.

Juraj Mavračić. 2025. Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents. arXiv:2510.24383.

Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. 2003. *Capability Myths Demolished*. Technical Report SRL2003-02. Systems Research Laboratory, Johns Hopkins University.

Andrew C. Myers and Barbara Liskov. 1997. A Decentralized Model for Information Flow Control. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP '97)*. 129-142. https://doi.org/10.1145/268998.266669

National Institute of Standards and Technology. 2023. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1

OASIS. 2013. *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard.

OWASP GenAI Security Project. 2025. OWASP Top 10 for LLM Applications 2025. https://genai.owasp.org/

Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527.

Niels Provos. 2003. Improving Host Security with System Call Policies. In *Proceedings of the 12th USENIX Security Symposium*.

Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.

Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. NIST Special Publication 800-207. https://doi.org/10.6028/NIST.SP.800-207

Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278-1308.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36*.

Fred B. Schneider. 2000. Enforceable Security Policies. *ACM Transactions on Information and System Security* 3, 1 (2000), 30-50. https://doi.org/10.1145/353323.353382

Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.

Sid Stamm, Brandon Sterne, and Gervase Markham. 2010. Reining in the Web with Content Security Policy. In *Proceedings of the 19th International Conference on World Wide Web (WWW '10)*. 921-930. https://doi.org/10.1145/1772690.1772784

AI Village and Grove Research. 2026. AI Swarm Dynamics Hackathon. Retrieved 11 October 2026 from https://swarmchasing.com/

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Lukas Weichselbaum, Michele Spagnuolo, Sebastian Lekies, and Artur Janc. 2016. CSP Is Dead, Long Live CSP! On the Insecurity of Whitelists and the Future of Content Security Policy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. 1376-1387. https://doi.org/10.1145/2976749.2978363

Mike West, Adam Barth, and Dan Veditz. 2016. Content Security Policy Level 2. W3C Recommendation. https://www.w3.org/TR/CSP2/

Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. Retrieved from https://simonwillison.net/2023/Apr/25/dual-llm-pattern/

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*.

A POLICY FOR THE AI VILLAGE REPLAY

Listing 2 is the policy used in Section 6.3 with two abbreviations, marked `...`: the network allowlist shows four of its 14 domains, and each argument rule shows its label but not its pattern. The complete file, `policies/ai-village.asp.json`, is in the artifact with every variant. Argument patterns are regular expressions applied to the canonical serialization of a shell command; `*:bash|cc:Bash` matches the village's shell tool and the coding

agent's shell tool. The variants in Sections 6.4 to 6.7 change only the fields named there, and the artifact contains each derived policy as a file.

Listing 2. Least-privilege policy for the AI Village replay, abbreviated (complete file: policies/ai-village.asp.json)

```
{
  "asp-version": "1.0",
  "description": "Least-privilege starting policy evaluated in report-only mode
  against AI Village replays. Decisions are counterfactual: the historical actions
  already happened.",
  "network": { "default": "deny",
    "allow": ["*.github.com", "gitlab.com", "theaidigest.org", "localhost", "..."] },
  "tools": { "default": "deny",
    "allow": ["village:*", "cc:Bash", "cc:Read", "cc:Grep", "cc:Glob",
      "cc:Write", "cc:Edit", "cc:WebSearch", "cc:WebFetch"],
    "deny": [],
    "deny-arguments": [
      { "tool": "*:bash|cc:Bash", "pattern": "...", "label": "remote script piped to
      shell" },
      { "tool": "*:bash|cc:Bash", "pattern": "...", "label": "recursive delete of
      root or home" },
      { "tool": "*:bash|cc:Bash", "pattern": "...", "label": "force push to shared
      history" },
      { "tool": "*:bash|cc:Bash", "pattern": "...", "label": "credential store
      access" },
      { "tool": "*:bash|cc:Bash", "pattern": "...", "label": "environment secret
      disclosure" }] },
  "content-trust": { "system-prompt": "high", "user-input": "medium", "tool-
  response": "low",
    "retrieved-document": "untrusted", "contaminated-write": "report", "contaminated-
    deny": [],
    "propagate-via-messages": false },
  "delegation": { "sub-agents": "inherit-and-restrict", "max-depth": 4 },
  "reporting": { "violations": "local:flight-recorder" },
  "swarm": { "window-seconds": 600, "write-cap": 12, "step-budget": 0,
    "repeated-intent": { "limit": 5, "min-agents": 3, "actions": ["tool", "message"]
    },
    "tripwires": { "fanout-children": 4, "shared-state-agents": 4, "consensus-min-
    cluster": 4,
      "consensus-density": 0.6, "echo-agents": 4, "echo-similarity": 0.6, "taint-
      agents": 0,
      "delegation-loop": false } }
}
```

B EVENT SEQUENCE OF THE INJECTION SCENARIO

Table 14 lists the 14 events of the injection scenario with the decision and rule for each under the full policy. With tripwire feedback off, events 12 and 13 change: the notifier's write is dropped by the contaminated-write rule, and the parser's second read is allowed.

Table 14. Injection scenario, decisions under the full policy with feedback on

#	Agent	Action	Target	Decision	Rule
1	orchestrator	spawn	receipt-parser	allow	policy.allow
2	orchestrator	spawn	ledger-agent	allow	policy.allow
3	orchestrator	spawn	notifier	allow	policy.allow
4	orchestrator	tool	search.read	allow	policy.allow

#	Agent	Action	Target	Decision	Rule
5	receipt-parser	tool	fs.read (untrusted)	allow	policy.allow
6	receipt-parser	tool	payments.transfer	drop	delegation.out_of_scope
7	receipt-parser	spawn	payment-helper	drop	delegation.scope_expansion
8	receipt-parser	message	ledger-agent	allow	policy.allow
9	ledger-agent	tool	ledger.read	allow	policy.allow
10	ledger-agent	tool	payments.transfer	drop	trust.contaminated_tool
11	receipt-parser	message	notifier	allow	policy.allow
12	notifier	tool	board.write	drop	tripwire.revoked
13	receipt-parser	tool	fs.read	drop	tripwire.revoked
14	orchestrator	tool	board.write	allow	policy.allow

C THREAT COVERAGE

Table 15 maps the threats of Section 3 to the controls that address them and to the evidence in this article.

Table 15. Threats, controls and evidence

Threat	Controls	Evidence	Status
Injection, single agent (A1)	Capability withdrawal; contaminated-write rule	Table 4; injection events 5, 10	Enforced on mediated paths
Injection spreading between agents (A1)	Taint propagation; injection-spread tripwire; revocation	Figure 2; injection events 8 to 13	Enforced on mediated paths; off for broadcast chat
Escalation through delegation (A2)	Inherit-and-restrict scopes; lineage admission; depth cap	Proposition 1; injection events 6, 7; swarm attack	Enforced
Tool-use policy violations (A2)	Deny list; default-deny allowlist; argument and network rules	Table 5 (97,472 would-block decisions)	Enforced; report-only on replay
Credential exposure (A2)	Argument rules; content inspection	Table 5 (4,681 credential-store reads)	Report-only on replay; pattern-based
Runaway loops and exhaustion (A3)	Step budget; write cap; repeated intent; loop and consensus tripwires	Table 3 (runaway); Table 7	Enforced; thresholds per deployment
Activity outside the gateway	Recorded as observed; feeds tripwires	Table 3 (observed events)	Detected only
Bypassing the gateway	Engine-owned dispatch; MCP proxy credentials; container confinement	MCP gateway test suite; Table 11 (16 of 16 probes closed when confined)	Holds only under the confinement condition
Confidential data in model requests (A4)	Confidentiality labels; egress monitor with engine identity, fail-closed resolution and revocation	Proposition 3; Tables 9 and 10	Enforced for requests routed through the egress monitor
Laundering taint through a fresh context (A1, A2)	Task-scoped taint; tasks opened only by a controller	Task-scoped taint test suite; Table 11 (I3, A2); Figure 4	Enforced under task scope